GRADIENT-FREE LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS FOR FEW-SHOT LEARNING

A PROJECT REPORT

Submitted by

TANYA M CHERRY

PRIYA DHARSHINI C

MUKESHRAJ S

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER TECHNOLOGY

ANNA UNIVERSITY, MIT CAMPUS

CHENNAI - 600 044

April 2026

BONAFIDE CERTIFICATE

Certified that this project report **"GRADIENT-FREE LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS FOR FEW-SHOT LEARNING"** is the bonafide work of Tanya M Cherry (2022503529), Priya Dharshini C (2022503501) & Mukeshraj S (2022503309) who carried out the project work under my supervision.

SIGNATURE

Dr. B. THANASEKHAR

SUPERVISOR

Professor

Department of Computer Technology

Anna University , MIT Campus

Chromepet - 600044

SIGNATURE

Dr. PONSY R.K SATHIA BHAMA

HEAD OF THE DEPARTMENT

Associate Professor

Department of Computer Technology

Anna University, MIT Campus

Chromepet - 600044

ABSTRACT

The number of parameters plays a vital role in determining the computational complexity of fine-tuning. Large Language Models (LLMs) have shown considerable success in various natural language processing tasks. However, fine-tuning these models for specific use cases is computationally expensive and requires a large memory footprint, making it difficult for many researchers and developers. Low-Rank Adaptation (LoRA) is a method that reduces the number of parameters for fine-tuning these large models by introducing low-rank matrices.

Derivative-Free Optimization (DFO) techniques provide an alternative approach to parameter optimization by eliminating the need for gradient computation. Instead of relying on backpropagation, DFO methods explore the parameter space through function evaluations, making them suitable for large-scale models where gradient computation is costly or infeasible. These techniques enable efficient optimization in scenarios involving limited computational resources.

The proposed system integrates DFO with LoRA to optimize the adaptation parameters efficiently. Experimental results show that the model achieves up to 88.75% accuracy while maintaining low memory usage and reduced execution time. Furthermore, the approach improves accuracy by 3.77% over gradient-based LoRA and outperforms Black-Box Tuning (BBT) by over 31%.

ACKNOWLEDGEMENT

We take this humble opportunity to thank **Dr. P. Jayashree**, Dean and Professor, MIT Campus, Anna University, and **Dr. Ponsy R K Sathia Bhama**, Associate Professor & Head, Department of Computer Technology, MIT Campus, Anna University for providing all the lab facilities in pursuit of this project.

Undertaking this project has helped us learn a lot, and we would like to express our sincere gratitude towards our Project Guide **Dr. B. Thanasekhar**, Professor, Department of Computer Technology, MIT, Anna University, whose kind guidance, and proper directions helped us in shaping this project perfectly.

We acknowledge the feedback of panel members **Dr. Ponsy R K Sathia Bhama**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr. R. Kathiroli**, Assistant Professor (Sr. Gr.), Department of Computer Technology, MIT, Anna University and **Dr. K. Kottilingam**, Assistant Professor, Department of Computer Technology, MIT, Anna University, in reviewing our work and encouraging us to successfully implement our project.

We thank all the teaching and non-teaching members of the Department of Computer Technology, MIT, Anna University, seniors, juniors, and friends whose appreciable help, either directly or indirectly helped in the smooth completion of the Creative and Innovative Project in a limited time frame. I would also like to thank my family for being striving force of support.

Tanya M Cherry (2022503529)
Priya Dharshini. C (2022503501)
Mukeshraj S (2022503309)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO
	ABSTRACT	**iii**
	LIST OF TABLES	**xi**
	LIST OF FIGURES	**xii**
	LIST OF ABBREVIATIONS	**xiv**
1	**INTRODUCTION**	1
	1.1 LARGE LANGUAGE MODELS (LLMS)	1
	1.2 FEW-SHOT LEARNING IN LLMS	2
	1.3 PARAMETER-EFFICIENT FINE-TUNING (PEFT) METHODS	2
	1.3.1 Prompt Tuning	3
	1.3.2 Low-Rank Adaptation (LoRA)	3
	1.4 LIMITATIONS OF GRADIENT-BASED OPTIMIZATION	3
	1.5 DERIVATIVE-FREE OPTIMIZATION (DFO)	4
	1.6 DERIVATIVE-FREE OPTIMIZATION FOR LoRA-BASED FINE-TUNING	4
	1.7 CHALLENGES IN EXISTING METHODS	5
	1.8 PROBLEM STATEMENT	5
	1.9 OBJECTIVES	6
	1.10 OUTLINE OF THE PROJECT REPORT	6

2	**LITERATURE SURVEY**	7
	2.1 LoRA-BASED FINE-TUNING OF LARGE LANGUAGE MODELS	7
	2.2 DERIVATIVE-FREE OPTIMIZATION METHODS	8
	2.3 EVOLUTIONARY AND CMA-ES BASED OPTIMIZATION	8
	2.4 FEW-SHOT LEARNING METHODS	9
	2.5 LoRA SURVEYS AND PARAMETER-EFFICIENT METHODS	9
	2.6 EFFICIENT LLM DESIGN AND OPTIMIZATION TECHNIQUES	10
	2.7 MULTI-MODEL SERVING AND SCALABLE SYSTEMS	10
	2.8 INFERENCES	11
3	**GRADIENT-FREE LoRA OPTIMIZATION OF FEW-SHOT PROMPTS**	12
	3.1 INTRODUCTION	12
	3.2 SYSTEM ARCHITECTURE AND OVERVIEW	13
	3.3 DATASET AND MODEL CONFIGURATION	15
	3.3.1 Dataset Preparation	15
	3.3.2 Model Configuration	16
	3.4 SYSTEM MODULES	17
	3.4.1 Dataset Collection and Few-Shot Preparation	17
	3.4.2 Base Model Selection and	18

Configuration

3.4.3 Low-Rank Adaptation (LoRA) Injection 18

3.4.4 Gradient-Free Optimization of LoRA Parameters 19

3.4.5 Intrinsic Dimensionality 20

3.4.6 Performance Evaluation and Analysis 20

3.5 SYSTEM DESIGN : MATHEMATICAL FORMULATION 21

3.5.1 Problem Formulation 21

3.5.2 Low-Rank LoRA Parameterization Inside Attention 22

3.5.3 Intrinsic Subspace and Projection Tensors 22

3.5.4 Layer-wise DFO Objective 23

3.5.5 Initialization of Projection Tensors 23

3.5.6 CMA-ES Formulation 24

3.5.7 Fireworks Algorithm Formulation (F-LoRA) 24

4 IMPLEMENTATION 26

4.1 PLATFORM USED 26

4.2 TOOLS USED 26

4.3 MODEL USED 27

4.4 DATASET PREPROCESSING AND FEW-SHOT PREPARATION 28

4.4.1 Pattern and Verbalizer Design 28

	4.4.2 Few-Shot Data Splitting	29
	4.4.3 Tokenization	29
	4.5 IMPLEMENTATION OF LOW-RANK ADAPTATION (LoRA) INJECTION	30
	4.5.1 Integration of Low-Rank Modules with Transformer Layers	30
	4.5.2 Injection into Self-Attention Mechanism	31
	4.5.3 Initialization of Low-Rank Parameters	31
	4.6 IMPLEMENTATION OF GRADIENT-FREE OPTIMIZATION	32
	4.6.1 Initialization of intrinsic vectors	32
	4.6.2 Projection Mapping to LoRA Parameters	33
	4.6.3 Layer-wise Optimization Strategy	33
	4.6.4 CMA-ES Implementation (C-LoRA)	34
	4.6.5 Fireworks Algorithm Implementation (F-LoRA)	35
	4.6.6 Overall Optimization Workflow	36
	4.7 EVALUATION METRICS	37
5	**RESULTS AND ANALYSIS**	39
	5.1 EXPERIMENTAL SETUP	39
	5.2 VERIFICATION OF TRAINABLE AND FROZEN PARAMETERS	39
	5.3 HIDDEN STATE STATISTICS COMPUTATION	40

	5.4 INITIALIZATION OF PROJECTION MATRICES	41
	5.5 PERFORMANCE RESULTS ACROSS DATASETS	41
	5.6 COMPARISON WITH GRADIENT-BASED LORA	42
	5.7 COMPARISON WITH BLACK-BOX TUNING (BBT)	43
	5.8 PERFORMANCE OF OTHER DERIVATIVE-FREE METHODS	44
	5.9 COMPARISON STUDY	45
6	**CONCLUSION AND FUTURE WORK**	46
	REFERENCES	47

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Model Parameters	16
4.1	Tools Used	26
4.2	RoBERTa Description	27
5.1	Performance Comparison of Proposed LoRA with DFO across Datasets	42
5.2	Comparison between Gradient-Based LoRA and Proposed Gradient-Free LoRA	43
5.3	Comparison between BBT and Proposed DFO-LoRA on SST-2 Dataset	43
5.4	Accuracy (%) of Manual Prompting and In-Context Learning on Benchmark Datasets	44
5.5	F1-Score (%) of Manual Prompting and In-Context Learning under Few-Shot Settings	44

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Proposed Architecture for Gradient-Free LoRA Optimization with Few-Shot Prompts	14
4.1	Pattern and Verbalizer-Based Input Construction	28
4.2	LoRA Injection into Self-Attention Layers	31
4.3	CMA-ES Based Optimization of Intrinsic LoRA Parameters	35
4.4	Fireworks Algorithm-Based Optimization for Low-Rank Parameter Adaptation	36
5.1	Verification of Trainable and Frozen Parameters in LoRA Implementation	40
5.2	Computation of Hidden State Statistics across Transformer Layers	40
5.3	Initialization of Projection Matrices using Layer-wise Hidden State Statistics	41
5.4	Performance Comparison of Different Methods on SST-2 Dataset	45
5.5	Performance and Efficiency Comparison between BBT and DFO-LoRA on SST-2 Dataset	45

LIST OF ABBREVIATIONS

BBT	–	Black-Box Tuning
C-LoRA	–	CMA-ES based LoRA
CMA-ES	–	Covariance Matrix Adaptation Evolution Strategy
DFO	–	Derivative-Free Optimization
F-LoRA	–	Fireworks Algorithm based LoRA
FWA	–	Fireworks Algorithm
ICL	–	In-Context Learning
LLM	–	Large Language Model
LoRA	–	Low-Rank Adaptation
MLM	–	Masked Language Model
PEFT	–	Parameter-Efficient Fine-Tuning
WQ, WK, WV	–	Query, Key, Value Weight Matrices

CHAPTER 1

INTRODUCTION

Large Language Models (LLMs) have become an important part of modern artificial intelligence, enabling applications such as text generation, translation, and summarization. They are trained on large amounts of text data and are capable of understanding and producing human-like language [14]. However, adapting these models to specific tasks requires fine-tuning, which can be computationally expensive and challenging, especially when only limited labeled data is available.

To address these challenges, parameter-efficient fine-tuning techniques such as Low-Rank Adaptation (LoRA) and prompt-based methods have been introduced to reduce the number of trainable parameters [6], [14]. However, most of these methods still rely on gradient-based optimization, which can increase memory usage and lead to instability. Recently, gradient-free optimization methods have emerged as an alternative by eliminating the need for backpropagation and instead optimizing through function evaluations [3], [9]. In this project, gradient-free optimization is applied to low-rank adaptation by injecting trainable modules into self-attention layers, enabling efficient and stable fine-tuning in few-shot learning scenarios [6], [11].

1.1 LARGE LANGUAGE MODELS (LLM)

LLMs are advanced artificial intelligence systems that use deep learning techniques to understand and generate human-like text. They are trained on vast amounts of data, including books, articles, and web pages, allowing them to capture complex language patterns and context [14]. LLMs can perform a wide range of tasks such as text generation, translation, summarization, and question answering. Popular models

such as GPT, BERT, and LLaMA demonstrate the effectiveness of LLMs in real-world applications. However, due to their large number of parameters, training and fine-tuning these models require significant computational resources, which has led to the development of efficient techniques such as Low-Rank Adaptation and advanced optimization methods [6], [14].

1.2 FEW-SHOT LEARNING IN LLM

Few-shot learning in LLMs is described as the ability of a model to accomplish a task based on a small number of input examples rather than needing a large amount of data to accomplish this task [11], [13]. In this type of learning, a small number of input prompts are given to the model to help it understand the pattern and generate the required output. With this type of learning, LLMs such as GPT are able to learn various tasks such as text classification, translation, or question answering without needing large amounts of data. This type of learning is quite useful for various AI applications [11], [13]

1.3 PARAMETER-EFFICIENT FINE-TUNING (PEFT) METHODS

The parameter-efficient fine-tuning (PEFT) methods are techniques used to fine-tune LLMs to fit specific tasks without retraining all parameters [6], [14]. Fine-tuning involves retraining all parameters of an LLM to fit a task. Since an LLM has billions of parameters, fine-tuning is computationally expensive and requires a lot of memory. The parameter-efficient fine-tuning techniques are efficient because they only fine-tune some parameters or add a small number of parameters to fine-tune an LLM without retraining all parameters [6], [14]. Some parameter-efficient fine-tuning methods include LoRA, Adapters, and Prefix Tuning.

1.3.1 Prompt Tuning

Prompt Tuning is a parameter-efficient fine-tuning method used in adapting LLMs for a particular task using a smaller set of prompt vectors that are learnable [14]. It does not fine-tune the entire model parameters; rather, a series of prompts are inserted into the input sequence to prompt the model to output a particular response. Prompt tuning is a computationally efficient method with low memory requirements and high performance. It can be used with GPT and BERT models [14].

1.3.2 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique that is employed to adapt Large Language Models by adding a number of small low-rank matrices to the selected layers of the model [1], [6], [8]. In the course of adapting the model, the original weights are frozen. Instead of updating all the weights of the model, the update of the weights is split into two smaller matrices. This technique is employed to adapt large models efficiently without compromising the performance of the model [1], [6], [8]. LoRA is largely employed in transformer-based models such as GPT and BERT.

1.4 LIMITATIONS OF GRADIENT-BASED OPTIMIZATION

Gradient-based optimization techniques are widely used for training and fine-tuning machine learning models by leveraging gradients of the loss function. However, in the context of LLMs, these methods can be computationally expensive due to the large number of parameters and model complexity. They are also prone to issues such as vanishing and exploding gradients, which can hinder effective training, and they significantly increase memory consumption during optimization [3], [15]. These challenges become even more critical in few-shot learning scenarios, where only a limited number of training examples are available, making it difficult to

compute stable and meaningful gradients and leading to poor generalization [11], [13]. As a result, these limitations have motivated the exploration of alternative approaches such as derivative-free optimization, which can operate without gradient information and are better suited for low-data and resource-constrained settings [3], [9].

1.5 DERIVATIVE-FREE OPTIMIZATION (DFO)

Derivative-Free Optimization (DFO) is a technique used for optimization, where the best solution for a given problem is found without calculating the gradients and derivatives of the objective function [3], [9], [15]. Instead, the objective function is directly used to find the best parameters for a solution. This technique is best used in situations where the gradients and derivatives cannot be easily calculated, such as in complex and large models. In the context of Large Language Models, the DFO technique is best used for optimizing parameters, where techniques such as LoRA are used, thus avoiding the use of other techniques and the problems associated with them [3], [6].

1.6 DERIVATIVE-FREE OPTIMIZATION FOR LORA-BASED FINE-TUNING

DFO is a technique that can be used to optimize parameters introduced by LoRA during fine-tuning [3], [6], [8]. In this technique, instead of computing gradients to update parameters, optimization is performed by evaluating various parameter configurations and selecting those that improve model performance. Since LoRA only introduces a small set of parameters to be optimized to the model, this optimization technique is computationally efficient [6], [8]. This optimization technique is efficient because it does not involve the computationally expensive task

of computing gradients to update parameters while still being effective in adapting the model to a task [3], [9].

1.7 CHALLENGES IN EXISTING METHODS

However, challenges still exist in efficient adaptation despite advancements in Large Language Models and parameter-efficient fine-tuning. Methods like Low-Rank Adaptation (LoRA) reduce parameters but still need proper optimization for stable performance [6], [8]. Derivative-free optimization avoids gradients but may face slow convergence [3], [9]. Moreover, many approaches treat fine-tuning and optimization separately, highlighting the need for efficient and stable methods in few-shot scenarios [6], [14].

1.8 PROBLEM STATEMENT

Fine-tuning Large Language Models for downstream tasks typically requires substantial computational resources and memory, as traditional approaches update all model parameters using gradient-based optimization. In few-shot learning scenarios, full fine-tuning often leads to overfitting and unstable performance. Although parameter-efficient fine-tuning methods such as prompt tuning, adapter tuning, and low-rank adaptation reduce the number of trainable parameters, they still depend on backpropagation and gradient computation, which increases memory usage and limits scalability. Existing gradient-free optimization methods primarily focus on prompt tuning and suffer from slow convergence and limited control over internal model representations. Therefore, there is a need for an efficient, stable, and memory-efficient fine-tuning approach that can adapt large language models in few-shot settings without relying on gradient-based optimization.

1.9 OBJECTIVES

- To design a parameter-efficient fine-tuning framework for large language models that avoids gradient-based optimization.
- To integrate low-rank adaptation modules into the self-attention layers of a pretrained language model.
- To propose and apply new gradient-free low-rank adaptation methods, namely C-LoRA and F-LoRA, for optimizing low-rank parameters in few-shot learning scenarios.
- To evaluate the proposed approach on benchmark datasets and compare its performance with existing gradient-based and gradient-free fine-tuning methods.
- To analyze the efficiency of the proposed method in terms of performance, memory usage, and convergence behavior.

1.10 OUTLINE OF THE PROJECT REPORT

This project report is structured into six chapters. Chapter 2 presents a comprehensive literature survey, reviewing existing methods related to low-rank adaptation, gradient-based and gradient-free optimization techniques, along with their limitations. Chapter 3 describes the proposed system, including the overall architecture, dataset preparation, LoRA injection, and the application of derivative-free optimization methods. Chapter 4 details the implementation aspects, including the tools, platform, model configuration, and step-by-step workflow of the system. Chapter 5 discusses the experimental results and performance analysis, comparing the proposed approach with existing methods in terms of accuracy, efficiency, and resource usage. Finally, Chapter 6 concludes the work and highlights potential directions for future research.

CHAPTER 2

LITERATURE SURVEY

This chapter provides an overview of existing research related to LoRA, LLMs, few-shot learning, and derivative-free optimization techniques. The literature is organized into different sections based on methodologies and their contributions to efficient model tuning, optimization, and scalable learning.

2.1 LoRA-BASED FINE-TUNING OF LARGE LANGUAGE MODELS

The HybridMoE [1] approach introduces a parameter-efficient fine-tuning method by combining a hybrid Mixture-of-Experts architecture with LoRA. It enhances multitask learning by selectively sharing low-rank matrices across tasks, which reduces the number of trainable parameters while preserving performance. However, it relies on gradient-based optimization, leading to higher computational cost and potential instability in low-resource settings. SecLoRA [2] presents a privacy-preserving LoRA tuning framework that improves convergence through local pre-optimization before secure training. It employs encryption techniques to safeguard sensitive data and model parameters. While it strengthens security and efficiency, it introduces additional computational and communication overhead. Revisiting LoRA [8] enhances the original LoRA approach by incorporating adaptive rank adjustment and attention enhancement mechanisms. These improvements boost model accuracy and convergence while keeping computational cost low. However, its performance depends heavily on proper parameter tuning and may vary across tasks. Cannikin [7] focuses on efficient deployment of multiple LoRA models using SLO-aware scheduling. It optimizes resource allocation and reduces latency in multi-model

environments. However, it increases system complexity due to scheduling and interference management.

2.2 DERIVATIVE-FREE OPTIMIZATION METHODS

A derivative-free optimization method [3] is proposed to address issues such as exploding and vanishing gradients. It eliminates the need for gradient computation by using a search-based optimization approach, improving robustness. However, the method is highly sensitive to hyperparameters and may get trapped in local optima. Compact model parameter extraction [15] utilizes derivative-free optimization to estimate model parameters efficiently without relying on gradient information. This approach reduces simulation cost and enhances robustness. However, it does not guarantee a global optimal solution and often requires multiple iterations. Benchmarking of derivative-free optimization algorithms [9] compares various methods under different conditions. The study shows that stochastic methods perform better for complex problems, while deterministic approaches are more efficient when fewer evaluations are needed. However, no single method consistently performs best across all scenarios.

2.3 EVOLUTIONARY AND CMA-ES BASED OPTIMIZATION

The reference vector-based CMA-ES method [4] converts constrained optimization problems into a multi-objective framework. This approach improves convergence and solution ranking, but it increases computational complexity and requires careful parameter tuning. Fast CMA-ES [10] enhances scalability by using low-rank covariance approximations. It reduces computational cost and accelerates convergence. However, approximation errors can impact performance, especially in high-dimensional settings. The matrix-free CMA-ES approach [12] avoids explicit covariance matrix computations, thereby reducing computational overhead.

However, it lacks adaptive step-size control and may require higher memory usage in more complex scenarios. The Chaotic Firework Algorithm [5] improves optimization by incorporating chaotic maps and perturbation strategies. It enhances convergence and helps escape local optima, but its effectiveness depends heavily on proper parameter tuning.

2.4 FEW-SHOT LEARNING METHODS

Few-shot learning aims to enable models to learn effectively from a limited number of data samples. A transformer-based method [11] introduces a unified Query-Support framework, where both support and query samples are modeled using attention mechanisms. This approach enhances feature representation and improves classification accuracy by combining global and local learning. However, it results in increased architectural complexity and requires careful training to achieve optimal performance. A comprehensive survey on few-shot learning [13] categorizes different methods based on variations in instances, classes, and data distributions in open-world settings. It highlights key challenges such as incomplete data, dynamic learning environments, and limitations in generalization. However, the study mainly emphasizes theoretical analysis and provides limited guidance for practical implementation in real-world applications.

2.5 LoRA SURVEYS AND PARAMETER-EFFICIENT METHODS

A survey on LoRA techniques [6] presents a structured overview of parameter-efficient fine-tuning methods for large language models. It categorizes different approaches based on aspects such as downstream adaptation, cross-task generalization, efficiency enhancement, and privacy preservation. The study emphasizes the benefits of LoRA in reducing computational cost while maintaining model performance. However, it primarily focuses on existing methods and provides

limited insight into alternative optimization strategies. Revisiting LoRA [8] further advances this area by enhancing low-rank adaptation through adaptive parameter tuning and improved attention mechanisms. This approach demonstrates better stability and generalization, although its effectiveness depends on task-specific configurations and careful parameter selection. Moreover, some recent papers investigate the application of dynamic rank adaptation and hybrid optimization techniques for further improving LoRA efficiency.

2.6 EFFICIENT LLM DESIGN AND OPTIMIZATION TECHNIQUES

A review on efficient LLM techniques [14] explores various strategies such as model compression, efficient fine-tuning, and optimized inference. It classifies these methods into parameter-centric, architecture-centric, training-centric, and data-centric approaches. These techniques effectively reduce computational cost while maintaining model performance. However, choosing the most appropriate method depends on hardware limitations and specific application needs. Cannikin [7] further enhances efficiency by optimizing resource allocation for multiple LoRA models through intelligent scheduling mechanisms. This approach improves system throughput and reduces latency, but it also introduces additional system complexity and dependence on workload patterns.

2.7 MULTI-MODEL SERVING AND SCALABLE SYSTEMS

Efficient management of multiple models is crucial for scalable deployment of large language models. Cannikin [7] introduces a scheduling-based system that allows concurrent serving of multiple LoRA models on shared GPU resources, improving throughput and resource utilization. However, its performance may decline under highly dynamic workloads. Benchmarking studies [9] emphasize the importance of selecting suitable optimization algorithms based on system conditions. They show

that performance varies with workload characteristics, highlighting the need for adaptive and scalable solutions in multi-model environments. Additionally, efficient LLM system designs [14] focus on scalable deployment through optimized inference pipelines and resource-aware strategies. These approaches enable effective handling of multiple models but require careful trade-offs between latency, cost, and accuracy.

2.8 INFERENCES

From the reviewed literature, it is clear that substantial progress has been made in improving the efficiency of large language models through parameter-efficient fine-tuning and optimization techniques. Methods such as LoRA effectively reduce computational cost and memory usage while maintaining strong performance. However, most existing approaches still rely on gradient-based optimization, which leads to scalability challenges and increased computational overhead, especially in resource-constrained environments. Derivative-free and evolutionary optimization methods offer an alternative by removing the need for gradient information and providing robustness in complex optimization scenarios. Despite these advantages, they require careful parameter tuning and do not always guarantee globally optimal solutions. Few-shot learning techniques help address limited data availability and enhance model generalization, but they introduce additional architectural complexity and training overhead. Likewise, system-level optimizations such as efficient scheduling, multi-model serving, and resource-aware deployment improve scalability and performance, but they also increase system complexity.

Overall, existing approaches indicate the need for more efficient, scalable, and adaptable optimization strategies that reduce reliance on gradient-based methods while maintaining high performance. These insights form the basis for the proposed work.

CHAPTER 3

GRADIENT-FREE LoRA OPTIMIZATION OF FEW-SHOT PROMPTS

This chapter provides an overview of existing research related to Low-Rank Adaptation (LoRA), Large Language Models (LLMs), few-shot learning, and derivative-free optimization techniques. The literature is organized into multiple sections based on different methodologies and their contributions to efficient model tuning, optimization, and scalable learning. These studies highlight both the strengths and limitations of current approaches in terms of computational efficiency, memory usage, and adaptability. They also offer insights into key challenges, including reliance on gradient-based optimization and scalability issues in resource-constrained environments. The analysis of these works helps identify existing research gaps and establishes the foundation for the proposed system.

3.1 INTRODUCTION

LLMs have shown strong performance in various natural language processing tasks, but adapting them to specific tasks remains challenging due to the high computational and memory cost of gradient-based fine-tuning. In few-shot learning scenarios, where only limited labeled data is available, full fine-tuning often leads to overfitting and unstable performance. Although parameter-efficient methods like LoRA reduce the number of trainable parameters, they still rely on backpropagation, which increases memory usage and limits scalability.

To overcome these limitations, this project focuses on gradient-free low-rank adaptation of LLMs. The approach injects low-rank matrices into the self-attention layers of a frozen pretrained model, ensuring that only a small set of parameters is

modified while keeping the original model weights unchanged. Instead of using gradients, the optimization is performed using derivative-free techniques, which rely only on forward-pass evaluations and eliminate the need for backpropagation.

Specifically, the project employs optimization methods such as Covariance Matrix Adaptation Evolution Strategy (CMA-ES) and the Fireworks Algorithm to update the low-rank parameters in an intrinsic low-dimensional space. This enables efficient exploration of the parameter space while reducing computational complexity. By optimizing each layer independently and controlling the evaluation budget, the method ensures stable convergence and better performance in few-shot settings.

Overall, the proposed method provides a memory-efficient and stable alternative to traditional fine-tuning techniques. It improves performance metrics such as accuracy and F1-score while reducing training time and memory usage, making it well-suited for resource-constrained environments and practical real-world applications of large language models.

3.2 SYSTEM ARCHITECTURE AND OVERVIEW

The architecture illustrates the **Gradient-Free Low-Rank Adaptation (DFO-LoRA)** framework, which is designed to efficiently adapt large language models such as RoBERTa in few-shot learning scenarios while reducing computational cost and memory usage. The framework performs parameter-efficient fine-tuning by injecting low-rank adaptation modules into the model and optimizing them without relying on gradient-based backpropagation.

The proposed framework follows a modular pipeline where the pretrained language model remains frozen, and only a small set of low-rank parameters are updated. Instead of updating the entire network, the system focuses on modifying specific

attention components, thereby improving efficiency and stability. The overall objective is to achieve effective model adaptation using minimal resources while maintaining high performance.

The proposed architecture diagram in Figure 3.1 includes all major modules of the project.

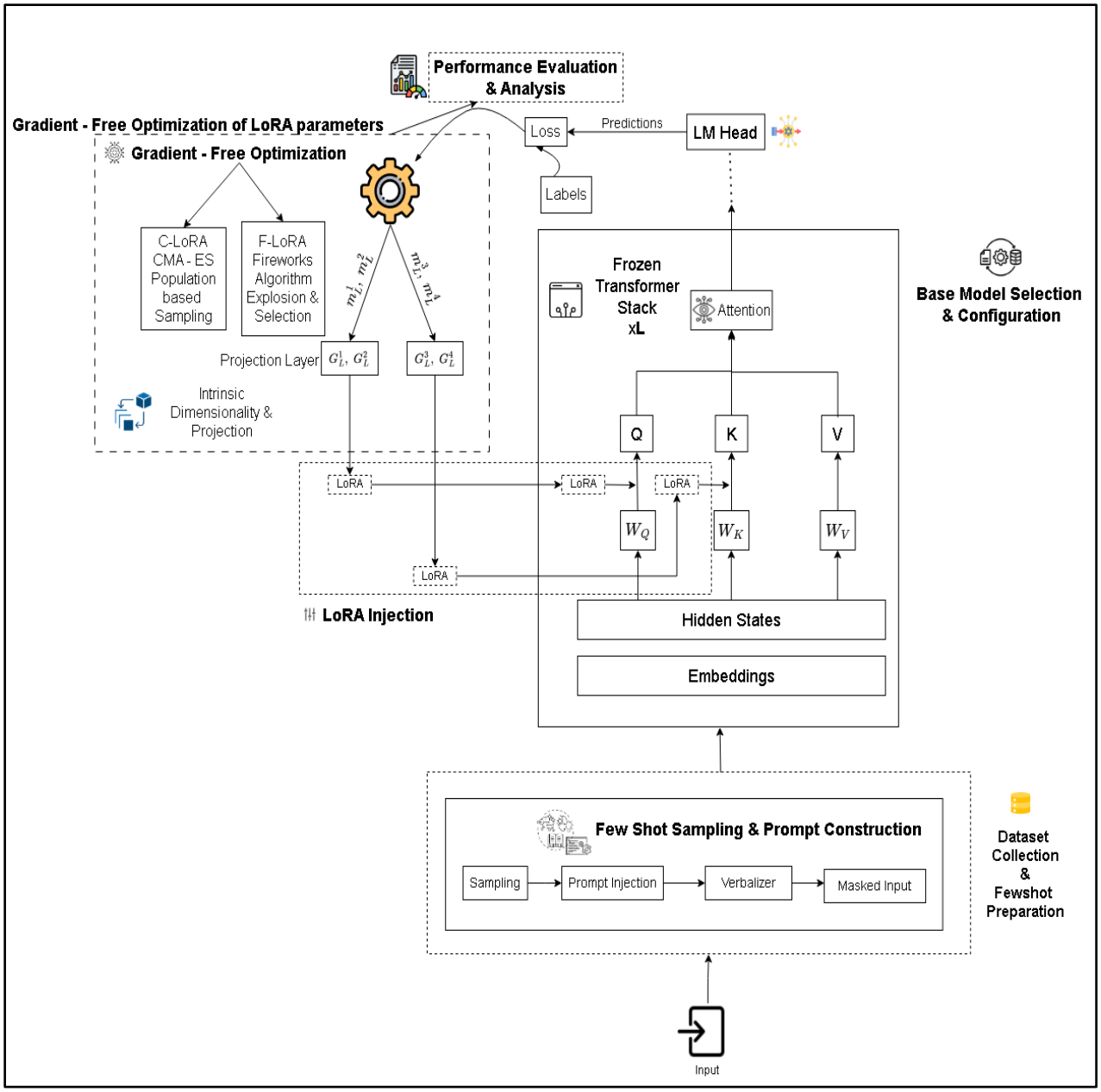


Figure 3.1 - *Proposed Architecture for Gradient-Free LoRA Optimization with Few-Shot Prompts*

3.3 DATASET AND MODEL CONFIGURATION

3.3.1 Dataset Preparation

We conduct experiments on standard Natural Language Understanding (NLU) benchmark datasets that cover multiple tasks such as sentiment analysis and topic classification. The datasets used in this work include SST-2 and Yelp for sentiment analysis, and AG News and DBpedia for topic classification, sourced from the Hugging Face dataset repository. These datasets are widely used to evaluate the performance of language models in few-shot learning scenarios.

The implementation of our method is based on the Hugging Face Transformers library and PyTorch framework. To simulate a realistic few-shot learning setup, we follow a true k-shot sampling strategy, where $k = 16$ samples are selected per class from the original training dataset. These samples are used to construct the few-shot training set DtrainD_{train}Dtrain and validation set DdevD_{dev}Ddev, ensuring that both sets are balanced and of equal size. The test set remains unchanged and is significantly larger than the training and validation sets to ensure reliable evaluation. For all NLU tasks, we adopt a prompt-based learning approach, where each input is transformed into a masked sentence using predefined patterns. A verbalizer mapping is defined to convert class labels into meaningful words that can be predicted by the model. The patterns and verbalizers used in this work are as follows:

SST-2 (Sentiment Analysis)

Pattern: ["text", "It", "was", "<mask>", "."]

Verbalizers: { "0": "bad", "1": "great" }

Yelp (Sentiment Analysis)

Pattern: ["text", "It", "was", "<mask>", "."]

Verbalizers: { "0": "bad", "1": "great" }

AGNews (Topic Classification)

Pattern: ["<mask>", "News:", "text"]

Verbalizers: { "0": "World", "1": "Sports", "2": "Business", "3": "Tech" }

DBPedia (Topic Classification)

Pattern: ["Category:", "<mask>", "text"]

Verbalizers: { "0": "Company", "1": "Education", "2": "Artist", "3": "Athlete", "4": "Office", "5": "Transport", "6": "Building", "7": "Natural", "8": "Village", "9": "Animal", "10": "Plant", "11": "Album", "12": "Film", "13": "Written" }

This prompt-based formulation allows the classification task to be reformulated as a masked language modeling problem, aligning with the pretrained objective of the language model. The prepared datasets are then used as input for the proposed gradient-free low-rank adaptation framework.

3.3.2 Model Configuration

The experiments were conducted using the **RoBERTa-Base model**, which consists of stacked self-attention layers and feed-forward networks designed for masked language modeling tasks.

TABLE 3.1 - *Model Parameters*

PARAMETERS	DETAILS
Model Type	Transformer Encoder
Hidden Layer Dimension	768
Number of Layers	12
Attention Heads	12

PARAMETERS	DETAILS
Head Dimension	64
Max Sequence Length	512
Pretraining Objective	Masked Language Modeling

The model was loaded using the Hugging Face Transformers library and executed on a GPU-enabled environment for efficient computation. All backbone parameters were frozen, and only the LoRA parameters were updated during optimization.

3.4 SYSTEM MODULES

The proposed system **Gradient-Free Low-Rank Adaptation (DFO-LoRA)** is organized into five modules, each responsible for a specific stage in the gradient-free low-rank adaptation pipeline. These modules work sequentially to ensure efficient processing, optimization, and evaluation of the model.

The proposed system comprises the following five modules:

1. Dataset Collection and Few-Shot Preparation
2. Base Model Selection and Configuration
3. Low-Rank Adaptation (LoRA) Injection
4. Gradient-Free Optimization of LoRA Parameters
5. Performance Evaluation and Analysis

3.4.1 Dataset Collection and Few-Shot Preparation

Benchmark datasets such as SST-2, Yelp, AGNews, and DBPedia are loaded using the Hugging Face Datasets library, ensuring efficient access to standardized data and streamlined preprocessing. A true k-shot sampling strategy (k = 16) is then applied

independently for each class to construct balanced training and validation sets, simulating a realistic few-shot learning scenario. The tasks are reformulated into a masked language modeling format using predefined prompt templates, where the input text is embedded into structured sentences containing a masked token to align with the pretrained model's objective. To enable classification, class labels are mapped to meaningful words known as verbalizers, which the model predicts at the masked position. Finally, baseline inference is performed using the frozen pretrained model without any adaptation, providing reference performance metrics for comparison with the proposed gradient-free low-rank adaptation approach.

3.4.2 Base Model Selection and Configuration

In this module, a pretrained transformer-based language model, RoBERTa-Base, is selected and loaded using the Hugging Face Transformers library. The model is configured to run in a GPU-enabled environment to ensure efficient computation. All backbone parameters are frozen to prevent gradient-based updates, allowing the model to retain its pretrained knowledge while reducing computational overhead. The internal architecture of the model is analyzed to identify key components, particularly the self-attention layers, including the Query (Q), Key (K), and Value (V) projection matrices. A forward-pass validation is performed to ensure that the model processes the prompt-based inputs correctly. This configuration prepares the model for efficient adaptation using low-rank modules without modifying the original weights.

3.4.3 Low-Rank Adaptation (LoRA) Injection

Low-Rank Adaptation (LoRA) is integrated into the pretrained transformer model to enable parameter-efficient fine-tuning. Instead of updating the entire model, low-rank matrices are injected into the self-attention layers, specifically into the Query

(Q) and Key (K) projection matrices. The weight updates are decomposed into two smaller matrices A and B with a predefined low rank (r = 8), significantly reducing the number of trainable parameters.

The original model weights remain frozen, and only the LoRA parameters are set as trainable, ensuring minimal computational overhead. These low-rank modules are applied across all transformer layers, allowing the model to capture task-specific patterns effectively while maintaining efficiency. Proper validation is performed to ensure that only LoRA parameters are updated during optimization. This approach enables scalable and memory-efficient adaptation of the language model without modifying its core architecture.

3.4.4 Gradient-Free Optimization of LoRA Parameters

The LoRA parameters are optimized using gradient-free techniques, eliminating the need for backpropagation and reducing memory consumption. Instead of directly optimizing the high-dimensional LoRA parameters, the process is carried out in a lower-dimensional intrinsic space. A compact latent vector is defined and mapped to the LoRA parameter space using projection matrices initialized from hidden-state statistics. This reduces the search space and improves optimization efficiency.

i) C-LoRA (CMA-ES based Optimization):

C-LoRA uses the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) to optimize the intrinsic latent vectors. It is a population-based method that samples multiple candidate solutions, evaluates them using forward-pass loss values, and updates the search distribution based on the best candidates. The covariance matrix is adapted iteratively to capture dependencies between variables, enabling efficient exploration and convergence toward optimal solutions without requiring gradient information.

ii) F-LoRA (Fireworks Algorithm based Optimization):

F-LoRA employs the Fireworks Algorithm (FWA), where each candidate solution generates multiple nearby solutions called sparks. These sparks explore both local and global regions of the search space. The number and spread of sparks depend on the fitness of the solution, allowing a balance between exploration and exploitation. Through repeated forward-pass evaluations, better solutions are identified and used to update the latent vectors, ensuring effective optimization without gradients.

3.4.5 Intrinsic Dimensionality

The concept of intrinsic dimensionality is used to represent the high-dimensional parameter space in a compressed form. A low-dimensional latent vector captures the essential variations required for adaptation and is projected into the LoRA parameters. This approach reduces computational complexity, enables faster convergence, and makes gradient-free optimization feasible for large models. Overall, this gradient-free optimization strategy enables efficient and stable tuning of LoRA parameters, particularly in few-shot learning settings.

3.4.6 Performance Evaluation and Analysis

The effectiveness of the proposed gradient-free low-rank adaptation approach is evaluated using standard performance metrics and efficiency measures. The optimized models, including Baseline, gradient-based LoRA, C-LoRA, and F-LoRA, are tested on benchmark datasets such as SST-2, Yelp, AGNews, and DBPedia to assess their performance in few-shot learning scenarios. The evaluation is primarily based on metrics such as **accuracy** and **F1-score**, which measure the model's classification performance.

In addition to performance metrics, the system also analyzes **efficiency parameters** such as training time, GPU memory usage, and convergence behavior. Memory consumption is monitored using tools like *nvidia-smi*, and training time is recorded for each method. Convergence trends are observed to evaluate the stability and effectiveness of the optimization process.

Finally, a comparative analysis is conducted between different approaches, including baseline inference, gradient-based LoRA, and the proposed gradient-free methods (C-LoRA and F-LoRA). The results demonstrate improvements in accuracy and F1-score along with reduced memory usage and training time, highlighting the effectiveness and scalability of the proposed method in resource-constrained environments.

3.5 SYSTEM DESIGN : MATHEMATICAL FORMULATION

This section presents the mathematical formulation of the proposed approach in a structured manner. It explains how the problem is defined and how the model is adapted using low-rank parameterization and derivative-free optimization techniques. The formulation focuses on im proving efficiency by optimizing only a subset of parameters while keeping the base model unchanged.

3.5.1 Problem Formulation

Let f be a pre-trained large language model (LLM) with frozen parameters θ. For an input sequence X and label Y, we define a loss

$$\mathcal{L}(f(.;X),Y) \tag{1}$$

In classical continuous prompt tuning, one directly optimizes prompt parameters $\rho \in \mathbb{R}^d$:

$$\rho \in \Theta$$

$$\rho^* = argmin\mathcal{L}(f(p;X),Y) \quad (2)$$

where Θ is the search space.

Black-box tuning (BBT) reduces dimension via a projection matrix $A \in \mathbb{R}^{D\times d}$ and optimizes a low-dimensional vector $z \in \mathbb{R}^d$:

$$z^* = \underset{z \in Z}{argmin}\mathcal{L}(f(A_z;X),Y), \quad (3)$$

with Z the search space for z.

The proposed method replaces prompt vectors with low-rank LoRA modules inside self attention and optimizes them by derivative free optimization (DFO) in an intrinsic low-dimensional space.

3.5.2 Low-Rank LoRA Parameterization Inside Attention

Consider a Transformer self-attention layer with original weight matrices

$$W_Q, W_K, W_V \in \mathbb{R}^{D\times k}, \quad (4)$$

where D is the hidden dimension and k is the query/key dimension.

LoRA introduces a low-rank update:

$$\Delta W = BA, \quad (5)$$

With

$$A \in \mathbb{R}^{r\times k}, B \in \mathbb{R}^{D\times r}, r \ll \min(D,K).$$

Low-rank modules are applied at least to W_Q and W_K:

$$W'_Q = W_Q + B_Q A_Q, \quad W'_K = W_K + B_K A_K. \quad (6)$$

Here A_Q, B_Q, A_K, B_K are learnable low-rank matrices, while W_Q, W_K remain frozen.

3.5.3 Intrinsic Subspace and Projection Tensors

Instead of optimizing full matrices directly, intrinsic vectors are used for each layer L:

$$m_L^1, m_{\mathrm{L}}^2, m_{\mathrm{L}}^3, m_{\mathrm{L}}^4 \in \mathbb{R}^d$$

where d is the intrinsic dimension.

Each intrinsic vector is mapped to a low-rank matrix via projection tensors:

$$G_L^1 \in \mathbb{R}^{r\times k\times d}, G_L^2 \in \mathbb{R}^{D\times r\times d}$$

$$G_L^3 \in \mathbb{R}^{r\times k\times d}, G_L^4 \in \mathbb{R}^{D\times r\times d}$$

The mapping is linear:

Where $G_{\mathrm{L}}^i\, m_{\mathrm{L}}^i$ denotes a tensor–vector multiplication that produces matrices of shapes $r \times k$ or $D \times r$.

3.5.4 Layer-wise DFO Objective

For each layer L and each intrinsic vector m_L^i $(with\ i = 1, \dots, 4)$, we define a derivative-free optimization problem:

$$m_L^{i*} = \underset{m_L^i \in \mathcal{M}}{argmin}\mathcal{L}(\, f(G_{\mathrm{L}}^i\, m_{\mathrm{L}}^i, X), Y) \qquad (7)$$

where $\mathcal{M}$ is the search space of intrinsic vectors, and f denotes the entire LLM with all LoRA modules generated from all intrinsic vectors $\{m_{\mathrm{L}}^i\}$. The method follows a divide and conquer scheme.

3.5.5 Initialization of Projection Tensors

The performance strongly depends on the initialization of G_{L}^i. Two strategies are considered:

1. **Simple normal initialization:**

$$G_{\mathrm{L}}^i \sim \mathcal{N}(0, 0.5^2).$$

This often leads to slower convergence and degraded performance.

2. **Hidden-state-aware initialization (used our the method):**

   Let $\hat{\sigma}$ be the empirical standard deviation of hidden states at layer L, σ_z the standard deviation of the DFO sampling distribution for intrinsic vectors m, and α a scalar. The standard deviation for G_{L}^{i} is defined as:

$$\sigma_m = \alpha\hat{\sigma}\sqrt{d}\sigma_z \tag{8}$$

   Then,

$$G_{\mathrm{L}}^{i} \sim \mathcal{N}(0, \sigma_m^2).$$

   Empirically, initialization using hidden-state statistics preserves model performance and accelerates convergence.

3.5.6 CMA-ES Formulation (C-LoRA)

For C-LoRA, the optimizer for each intrinsic vector m_{L}^{i} is the Covariance Matrix Adaptation Evolution Strategy (CMA-ES).

Sampling Step

For population size λ, sample candidate solutions:

$$m_j^{(t)} = \mu^{(t)} + \sigma^{(t)}\mathcal{N}\left(0, C^{(t)}\right), \qquad j = 1, \ldots., \lambda. \tag{9}$$

For each candidate, build the corresponding low-rank matrices through $G_{\mathrm{L}}^{i}\, m_{\mathrm{j}}^{(t)}$, update $\mathrm{W'_Q}$ and $\mathrm{W'_K}$, run the model, and compute fitness:

$$f_j^{(t)} = \mathcal{L}\left(f\left(G_{\mathrm{L}}^{i}\, m_{\mathrm{j}}^{(t)}, X\right), Y\right). \tag{10}$$

Further, selection and recombination, covariance adaptation, step size adaptation is done.

3.5.7 Fireworks Algorithm Formulation (F-LoRA)

For F-LoRA, each intrinsic vector m_{L}^{i} is optimized using a Fireworks Algorithm variant.

Representation

Each firework is a candidate solution m ∈ $\mathbb{R}^d$. At iteration t, we maintain a population:

$$\left\{m_1^{(t)}, \dots, m_N^{(t)}\right\},$$

where N is the population size.

Explosion Operator:

For a firework $m_{\mathrm{j,s}}^{(t)}$, generate S_j sparks in its neighborhood:

$$m_{\mathrm{j,s}}^{(t)} = m_{\mathrm{j}}^{(t)} + \Delta_{\mathrm{j,s}}^{(t)}, \;\; s = 1, \dots, S_j, \tag{11}$$

where $\Delta_{\mathrm{j,s}}^{(t)}$ is a random vector sampled within an explosion amplitude range (e.g., uniform in [−Aj,Aj]).

Better fireworks get smaller amplitudes and more sparks to refine good regions; worse ones get larger amplitudes and fewer sparks to explore.

Fitness Evaluation

For all sparks, construct the corresponding LoRA matrices via $G_{\mathrm{L}}^{i}\, m_{\mathrm{j,s}}^{(t)}$, update the attention weights, run the model, and compute loss:

$$f_{j,s}^{(t)} \;=\; \mathcal{L}\Big(\, f\left(G_{\mathrm{L}}^{i}\, m_{\mathrm{j,s}}^{(t)}, X\right), Y\Big). \tag{12}$$

The proposed approach adapts large language models using low-rank LoRA parameterization with derivative-free optimization, reducing computational cost while maintaining strong performance. It updates only a small subset of parameters in a low-dimensional space, and uses projection tensors with layer-wise optimization for structured and scalable learning without gradients. Additionally, improved initialization enables faster convergence and better stability, making it a practical and resource-efficient alternative to traditional fine-tuning.

CHAPTER 4

IMPLEMENTATION

This chapter outlines the implementation of the proposed Gradient-Free Low-Rank Adaptation framework for Large Language Models. It covers the tools, techniques, and procedures used for dataset preparation, LoRA integration, and gradient-free optimization. The implementation is designed to ensure efficient and scalable model adaptation, particularly in few-shot learning scenarios.

4.1 PLATFORM USED

The implementation of the proposed system is carried out using **Google Colab**, which provides a cloud-based development environment for executing Python code. It offers access to GPU acceleration, enabling efficient training and evaluation of large language models. The platform also supports integration with libraries such as Hugging Face Transformers and PyTorch, making it suitable for implementing and testing the gradient-free low-rank adaptation framework.

4.2 TOOLS USED

Table 4.1 lists the key Python Libraries used

TABLE 4.1 - *Tools Used*

TOOL	PURPOSE
PyTorch	Core deep learning framework for loading and running LLMs
Hugging Face Transformers	Access to pre-trained Large Language Models

TOOL	PURPOSE
Datasets	Loading and handling datasets
NumPy	Efficient numerical operations for optimization and vector handling
SciPy	Used for gradient-free optimization techniques (e.g., CMA-ES)
Matplotlib	Visualization of training results, accuracy, and memory usage
Seaborn	Enhanced plotting for performance comparison graphs
CUDA	GPU acceleration for faster model execution

4.3 MODEL USED

Table 4.2 provides an overview of the key features of RoBERTa Base model.

***TABLE 4.2** – RoBERTa Description*

FEATURE	DESCRIPTION
Model	RoBERTa-base
Type	Pre-trained transformer-based bidirectional encoder (masked language model)
Usage	Used as the base frozen LLM where only low-rank

FEATURE	DESCRIPTION
	prompt parameters are optimized using gradient-free methods
Training Strategy	Pre-trained using masked language modeling on large-scale corpora
Role in Optimization	Model weights remain fixed, only intrinsic prompt vectors are updated

4.4 DATASET PREPROCESSING AND FEW-SHOT PREPARATION

This phase outlines the preprocessing steps and data preparation techniques employed to adapt the dataset for efficient training under a few-shot learning setting.

4.4.1 Pattern and Verbalizer Design

To ensure compatibility with the pre-training objective of transformer-based language models, the classification task is reformulated as a masked language modeling problem. Each input instance is embedded within a predefined prompt template, referred to as a *pattern*, which includes a special <mask> token indicating the prediction position.

	label	verbalizer	text_preview	mlm_text_preview
0	0	bad	exactly how genteel and unsurprising the execution turns out to be ...	exactly how genteel and unsurprising the execution turns out to be It was <mask>....
1	0	bad	an all-time low for kevin costner ...	an all-time low for kevin costner It was <mask>....
2	0	bad	several cliched movie structures : ...	several cliched movie structures : It was <mask>....
3	0	bad	america 's knee-jerk moral sanctimony ...	america 's knee-jerk moral sanctimony It was <mask>....
4	0	bad	a dumb comedy ...	a dumb comedy It was <mask>....
5	0	bad	the plan to make enough into ` an inspiring tale of survival wrapped in the heart-pounding suspense ...	the plan to make enough into ` an inspiring tale of survival wrapped in the heart-pounding suspense of a stylish psychological thriller ' has flopped ...

Figure 4.1 - *Pattern and Verbalizer-Based Input Construction*

4.4.2 Few-Shot Data Splitting

To evaluate the model under limited supervision, a few-shot learning strategy is adopted. Instead of utilizing the entire dataset, only a small subset of labeled instances is selected for training. Specifically, a fixed number of samples ($k=16$) are drawn from each class to construct a balanced and representative training set.

The dataset is partitioned into three subsets:

- **Training set (D_{train}):** Contains a limited number of labeled examples per class
- **Validation set (D_d_{ev}):** Used for model tuning and early stopping, typically of equal size to the training set
- **Test set (D_{test}):** Retained in its original size to ensure reliable performance evaluation

This few-shot setup allows for assessing the model's generalization capability when trained on minimal data, which is a critical requirement for real-world applications with limited labeled resources.

4.4.3 Tokenization

Following prompt construction, the textual inputs are processed using a tokenizer corresponding to the pre-trained language model. Tokenization converts raw text into a sequence of subword tokens that can be interpreted by the model.

The tokenization process involves:

- Splitting text into subword units
- Mapping tokens to their corresponding numerical identifiers
- Inserting special tokens required by the model architecture (e.g., classification and separator tokens)
- Applying padding or truncation to maintain uniform input length
- Generating attention masks to differentiate between valid tokens and padded elements

The resulting tokenized representations are then used as inputs to the model during both training and inference. This step ensures consistency in input formatting and enables efficient utilization of the transformer architecture.

4.5 IMPLEMENTATION OF LOW-RANK ADAPTATION (LoRA) INJECTION

This section describes the implementation of Low-Rank Adaptation (LoRA) integrated with derivative-free optimization for efficient fine-tuning of large language models. The method injects trainable low-rank modules into the self-attention layers of the transformer while keeping the original model parameters frozen.

4.5.1 Integration of Low-Rank Modules into Transformer Layers

The core idea of LoRA is to introduce additional trainable matrices into the self-attention mechanism of the transformer. Instead of directly updating the original weight matrices, the method decomposes the weight update into two low-rank matrices.

For a given weight matrix W, the update is expressed as:

$$W' = W + BA$$

where:

- $\mathrm{A} \in \mathbb{R}^{r \times k}$
- $\mathrm{B} \in \mathbb{R}^{d \times r}$
- $r \ll d\ (low - rank\ constraint)$

These low-rank matrices are injected into key components of the self-attention mechanism, particularly the query (Q) and key (K) projections. The original weights W remain frozen, and only the parameters of A and B are optimized.

4.5.2 Injection into Self-Attention Mechanism

The low-rank modules are incorporated into each transformer layer's self-attention block. Specifically, the weight matrices corresponding to query and key projections are modified as:

$$W_Q = W_Q + B_Q A_Q$$
$$W_K = W_K + B_K A_K$$

```
Freezing all original model parameters

Injecting LoRA (rank=8, alpha=16.0) into attention layers
   Target modules: ['query', 'key']

Enabling gradients for LoRA parameters only...

LoRA Injection Statistics:
   Total layers: 12
   LoRA-injected layers: 12
   Original parameters: 124,645,632
   LoRA parameters: 294,912
   Trainable parameters: 294,912
   Trainable %: 0.24%

✓ LoRA injection complete!
```

Figure 4.2 - *LoRA Injection into Self-Attention Layers*

This selective injection ensures that the model captures task-specific information while maintaining the original knowledge encoded in the pre-trained model. Experimental observations indicate that applying LoRA to the query and key matrices yields better performance compared to other combinations.

4.5.3 Initialization of Low-Rank Parameters

The initialization of low-rank modules plays a critical role in model performance and convergence. Two strategies are considered:

- Random Initialization: Using a normal distribution

- Data-driven Initialization: Based on the distribution of hidden states from each transformer layer

Initialization using hidden state statistics is preferred, as it stabilizes training and improves convergence speed.

4.6 IMPLEMENTATION OF GRADIENT-FREE OPTIMIZATION

Algorithm 1 DFOs for Low-Rank Adaptation

Require: L-layer language model f

Budget of API calls B

Optimizers $\{O_i\}_{i=1}^{L}$

Loss function $\mathcal{L}$

1: Hidden variable: m_L^i, $i = 1, 2, 3, 4$

2: Random projections: G_L^i, $i = 1, 2, 3, 4$

3: Low-rank modules of each layer: A and B

4: **repeat**

5: **for** each hidden layer **do**

6: Evaluate: loss = $\mathcal{L}(f\ (G_L^i\ m_L^i))$

7: Update m_L^i by DFOs: $O_L^i(m_L^i, \text{loss})$

8: Update low-rank modules A and B through $G_L^i\ m_L^i$ while keeping G_L^i frozen

9: **end for**

10: **until** B / L times

4.6.1 Initialization of intrinsic vectors

Instead of directly optimizing high-dimensional model parameters, the method introduces low-dimensional intrinsic vectors for each transformer layer. These vectors define a compact search space for optimization.

For each layer *L*, a set of intrinsic vectors is defined:

$$m_L^1, m_L^2, m_L^3, m_L^4 \in \mathbb{R}^d$$

where *d* represents the intrinsic dimensionality.

The vectors are initialized using Layer-wise hidden state statistics.

Initialization based on hidden state distributions improves stability and convergence speed, especially in few-shot settings .

4.6.2 Projection Mapping to LoRA Parameters

The intrinsic vectors are projected into low-rank matrices using predefined mapping functions. These mappings transform low-dimensional vectors into parameters compatible with the transformer architecture.

The projection is defined as:

$$A_Q = G_L^1 m_L^1,\ B_Q = G_L^2 m_L^2$$
$$A_k = G_L^3 m_L^3,\ B_k = G_L^4 m_L^4$$

where G^i_L are projection matrices.

The resulting matrices are injected into the self-attention mechanism, enabling efficient parameter updates in a reduced-dimensional space.

4.6.3 Layer-wise Optimization Strategy

Require:

Pretrained L-layer transformer model

Intrinsic vectors m^l for each layer

Projection functions G^l

Gradient-free optimizer O

Loss function $\mathcal{L}$

Optimization budget B

1: Initialize intrinsic vectors m^l for all layers

2: **repeat**

3: **for** each layer l = 1 to L **do**

4: Compute forward pass using current parameters

5: Evaluate loss = $\mathcal{L}(f\ (G_{\mathrm{L}}^{i}\ m_{\mathrm{L}}^{i}))$

6: Update intrinsic vector m^l using optimizer $O_L^i(m_{\mathrm{L}}^{i}$, loss)

7: Map updated m^l to low-rank parameters $(A^l\ , B^l)$

8: Inject updated parameters into layer l

9: end for

10: **until** optimization budget B is reached

4.6.4 CMA-ES Implementation (C-LoRA)

Require:

Intrinsic vector m^l

Population size N

Mean μ and covariance Σ

Loss function $\mathcal{L}$

Optimization budget B

1: Initialize mean μ and covariance Σ

2: Initialize population of candidate solutions $\{m_1^l, m_2^l, \ldots\ldots, m_n^l\}$

3: **repeat**

4: Sample candidate solutions from distribution:

$m^l \sim \mathcal{N}(\mu, \Sigma)$

5: Evaluate each candidate using loss: $\mathcal{L}(f\ (G^l\ m^l\ ))$

6: Select best-performing candidates

7: Update mean μ based on selected candidates

8: Update covariance Σ to adapt search direction

9: **until** convergence or budget B is reached

```
──────────────────────────────────────────────────────────────
Optimizing Layer 0 with CMA-ES
──────────────────────────────────────────────────────────────
 Iterations per vector: 12
 Population size: 8

============================================================
 Optimizing m1 (intrinsic vector 1/4)
============================================================
 Iter   1/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832
 Iter   5/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832
 Iter  10/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832

 ✓ m1 complete
 Final loss: 3.4832
 Time: 36.1s
```

Figure 4.3 - *CMA-ES Based Optimization of Intrinsic LoRA Parameters*

4.6.5 Fireworks Algorithm Implementation (F-LoRA)

Require:

Intrinsic vector m^l

Number of fireworks N

Number of sparks S

Loss function $\mathcal{L}$

Optimization budget B

1: Initialize candidate solutions (fireworks) $\{m_1^l, m_2^l, \ldots\ldots, m_n^l\}$

2: **repeat**

3: **for** each firework m_i^l **do**

4: Generate sparks around m^l_i

5: Evaluate fitness of sparks using $\mathcal{L}(f\ (G^l\ m^l\ ))$

6: end for

7: Select best-performing candidates

8: Replace poor solutions using loser-out strategy

9: **until** convergence or budget B is reached

```
============================================================
 Optimizing m2 (intrinsic vector 2/4)
============================================================
 Iter   1/12 | Best: 3.8917 | Avg: 3.9699 | Min: 3.8917
 Iter   5/12 | Best: 3.8787 | Avg: 3.8845 | Min: 3.8787
 Iter  10/12 | Best: 3.8673 | Avg: 3.8720 | Min: 3.8673

 ✓ m2 complete
 Final loss: 3.8621
 Time: 75.4s
```

Figure 4.4 - *Fireworks Algorithm-Based Optimization for Low-Rank Parameter Adaptation*

4.6.6 Overall Optimization Workflow

The overall optimization workflow begins by initializing intrinsic vectors for all transformer layers, which define a low-dimensional search space. These vectors are optimized in a layer-wise manner and mapped to low-rank parameters, which are then injected into the self-attention layers. The model performance is evaluated, and the intrinsic vectors are updated using gradient-free methods such as CMA-ES or

the Fireworks Algorithm. This process is repeated iteratively until convergence or the optimization budget is reached.

4.7 EVALUATION METRICS

The performance of the proposed gradient-free low-rank adaptation framework is evaluated using standard classification metrics along with efficiency measures.

1. Accuracy

Accuracy measures the proportion of correctly predicted samples out of the total number of samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where *TP* (True Positives), *TN* (True Negatives), *FP* (False Positives), and *FN* (False Negatives) represent the prediction outcomes.

2. Precision

Precision measures how many of the predicted positive samples are actually correct:

$$Precision = \frac{TP}{TP + FP}$$

3. Recall

Recall measures how many of the actual positive samples are correctly identified:

$$Recall = \frac{TP}{TP + FN}$$

4. F1-Score

F1-Score is the harmonic mean of precision and recall, providing a balanced evaluation:

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

5. Efficiency Metrics

In addition to classification performance, the system is evaluated based on efficiency parameters:

- Training Time: Total time required for model optimization.
- Memory Usage: GPU memory consumed during training, monitored using tools like *nvidia-smi*.

The evaluation metrics offer a complete assessment of the proposed gradient-free low-rank adaptation framework by covering both performance and efficiency aspects. The metrics include key measures such as accuracy, precision, recall, and F1-score, which effectively evaluate the classification capability of the framework. This ensures that the model produces reliable and well-balanced classification results across different scenarios. In addition, efficiency metrics such as training time and memory usage highlight the computational advantages of the approach. These measures demonstrate that the framework requires fewer resources while maintaining strong performance. Furthermore, the inclusion of both performance and efficiency metrics provides a holistic evaluation of the system. Overall, the results confirm that the gradient-free low-rank adaptation framework achieves both high accuracy and efficient training, making it suitable for large-scale and resource-constrained environments.

CHAPTER 5

RESULTS AND ANALYSIS

This chapter presents the experimental results and analysis of the proposed gradient-free low-rank adaptation approach. It evaluates performance across multiple datasets and compares it with existing methods in terms of accuracy, efficiency, and resource usage.

5.1 EXPERIMENTAL SETUP

The experimental setup was implemented using PyTorch and the Hugging Face framework on a GPU-enabled environment (Colab CUDA). Experiments were conducted under a true few-shot setting, where $k=16$ samples per class were selected to form balanced training and validation sets $(|D_{train}| = |D_{dev}|)$, while retaining a larger test set for evaluation. The base model used was RoBERTa-base with all backbone parameters frozen. Low-Rank Adaptation (LoRA) was applied to the query and key projection matrices with a rank $r=8$, and optimization was performed in a low-dimensional intrinsic space of size $d=100$. Gradient-free optimization techniques, namely CMA-ES and the Fireworks Algorithm, were employed with a population size of 8 and a fixed evaluation budget of 400 per layer. The model was trained with a batch size of 8 and sequence length of 128, and performance was evaluated using accuracy and F1-score metrics.

5.2 VERIFICATION OF TRAINABLE AND FROZEN PARAMETERS

This step ensures that the Low-Rank Adaptation (LoRA) mechanism is correctly implemented by verifying that only the injected low-rank parameters are set as trainable, while all original model parameters remain frozen.

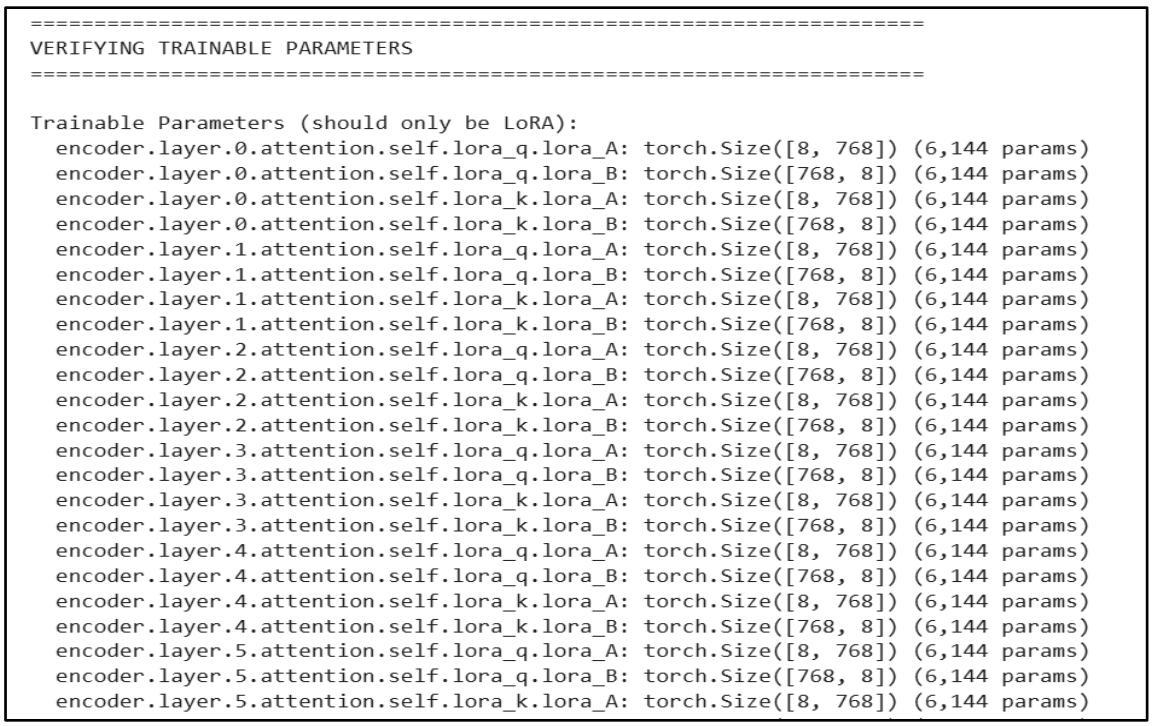


Figure 5.1 - *Verification of Trainable and Frozen Parameters in LoRA Implementation*

5.3 HIDDEN STATE STATISTICS COMPUTATION

The hidden state statistics were computed for all 12 transformer layers to capture the distribution of activations within the model. For each layer, the mean and standard deviation of the hidden representations were calculated, which serve as important indicators of the model's internal feature distribution.

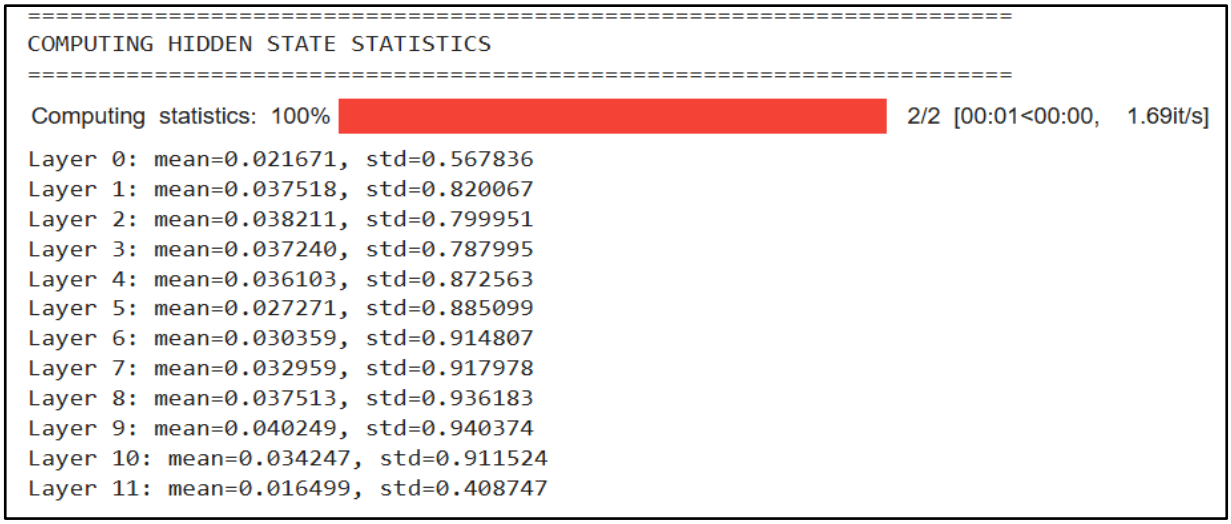


Figure 5.2 - *Computation of Hidden State Statistics across Transformer Layers*

5.4 INITIALIZATION OF PROJECTION MATRICES

For every layer, four projection matrices *(G_1, G_2, G_3, G_4)* corresponding to query and key adaptations were created with dimensions aligned to the model architecture (ie; 8 × 768 × 100 and 768 × 8 × 100). The standard deviation for initialization was derived from the respective layer's hidden state statistics, allowing the projection matrices to reflect layer-specific characteristics.

```
--------------------------------------------------------------------------
INITIALIZING RANDOM PROJECTION MATRICES
==========================================================================
Layer 0:
  G1 (A_Q): torch.Size([8, 768, 100]), σ=0.056784
  G2 (B_Q): torch.Size([768, 8, 100]), σ=0.056784
  G3 (A_K): torch.Size([8, 768, 100]), σ=0.056784
  G4 (B_K): torch.Size([768, 8, 100]), σ=0.056784
Layer 1:
  G1 (A_Q): torch.Size([8, 768, 100]), σ=0.082007
  G2 (B_Q): torch.Size([768, 8, 100]), σ=0.082007
  G3 (A_K): torch.Size([8, 768, 100]), σ=0.082007
  G4 (B_K): torch.Size([768, 8, 100]), σ=0.082007
Layer 2:
  G1 (A_Q): torch.Size([8, 768, 100]), σ=0.079995
  G2 (B_Q): torch.Size([768, 8, 100]), σ=0.079995
  G3 (A_K): torch.Size([8, 768, 100]), σ=0.079995
  G4 (B_K): torch.Size([768, 8, 100]), σ=0.079995
Layer 3:
  G1 (A_Q): torch.Size([8, 768, 100]), σ=0.078800
  G2 (B_Q): torch.Size([768, 8, 100]), σ=0.078800
  G3 (A_K): torch.Size([8, 768, 100]), σ=0.078800
  G4 (B_K): torch.Size([768, 8, 100]), σ=0.078800
Layer 4:
  G1 (A_Q): torch.Size([8, 768, 100]), σ=0.087256
  G2 (B_Q): torch.Size([768, 8, 100]), σ=0.087256
  G3 (A_K): torch.Size([8, 768, 100]), σ=0.087256
  G4 (B_K): torch.Size([768, 8, 100]), σ=0.087256
```

Figure 5.3 - *Initialization of Projection Matrices using Layer-wise Hidden State Statistics*

5.5 PERFORMANCE RESULTS ACROSS DATASETS

The results in Table 5.1 show that the proposed method achieves accuracy values

ranging from **81.22% to 88.75%** and F1-scores between **80.72% and 88.55%** across different datasets. The execution time varies from **37.41 to 45.96 minutes**, while memory usage ranges between **2276 MB and 2878 MB**. Among the datasets, DBPedia records the highest accuracy and F1-score, while Yelp achieves the lowest execution time.

TABLE 5.1 - *Performance Comparison of Proposed LoRA with DFO across Datasets*

DATASET	ACCURACY (%)	F1-SCORE (%)	TIME (MIN)	MEMORY (MB)
SST-2	81.22	80.72	42.75	2878
Yelp	87.09	86.79	37.41	2419
AGNews	82.84	82.34	45.96	2693
DBPedia	88.75	88.55	44.30	2276

5.6 COMPARISON WITH GRADIENT-BASED LORA

The comparison in Table 5.2 shows that the proposed Gradient-free LoRA achieves a 3.77% improvement in test accuracy and a 4.70% increase in F1-score compared to Gradient-based LoRA, while also reducing memory usage on the SST2 Dataset. This improvement is achieved by optimizing only low-rank parameters using gradient-free methods in a reduced intrinsic space while keeping the base model frozen, thereby enhancing efficiency and reducing memory overhead.

TABLE 5.2 - *Comparison between Gradient-Based LoRA and Proposed Gradient-Free LoRA*

METHOD	ACCURACY(%)	F1-SCORE(%)	MEMORY (MB)
LoRA with Adam	77.45	76.02	2983
Proposed LoRA with DFO	81.22	80.72	2878

5.7 COMPARISON WITH BLACK-BOX TUNING (BBT)

Table 5.3 demonstrates that DFO-LoRA substantially improves performance over BBT on the SST-2 dataset, with a 31.22% gain in accuracy, along with reduced computation time and memory consumption.

This improvement is achieved by optimizing low-rank parameters within self-attention layers using layer-wise gradient-free methods, enabling more effective internal representation learning than prompt-only tuning in BBT while reducing computational overhead.

TABLE 5.3 - *Comparison between BBT and Proposed DFO-LoRA on SST-2 Dataset*

METHOD	ACCURACY (%)	F1-SCORE (%)	TIME (MIN)	MEMORY (MB)
BBT on SST-2	50	32.31	63.83	3146
DFO-LoRA on SST-2	81.22	80.72	42.75	2878

5.8 PERFORMANCE OF OTHER DERIVATIVE-FREE METHODS

The results in Table 5.4 and Table 5.5 show that Manual Prompting achieves accuracy values in the range of **65%–76%** across most datasets, while In-Context Learning shows more variation, with values dropping as low as **11.65%–49.20%** for certain tasks. Manual Prompting performs consistently better on SST-2, Yelp, and AGNews, whereas In-Context Learning shows comparatively lower performance, especially on SST-2 and AGNews. These values indicate that prompt-based methods exhibit inconsistent performance across datasets, particularly in few-shot settings. Our proposed method outperforms both approaches by achieving higher and more stable performance across all datasets.

***Table 5.4** – Accuracy (%) of Manual Prompting and In-Context Learning on Benchmark Datasets*

Method	SST-2	Yelp	AGNews	DBPedia
Manual Prompt	76.72%	73.88%	67.09%	31.92%
In-context Learning	49.20%	70.76%	25.80%	46.04%

***Table 5.5** – F1-Score (%) of Manual Prompting and In-Context Learning under Few-Shot Settings*

Method	SST-2	Yelp	AGNews	DBPedia
Manual Prompt	76.65%	72.66%	65.51%	29.72%
In-context Learning	32.57%	69.58%	11.65%	42.24%

5.9 COMPARISON STUDY

Figure 5.4 illustrates the accuracy and F1-score of different methods on the SST-2 dataset, showing that DFO-LoRA achieves the highest performance among all approaches.

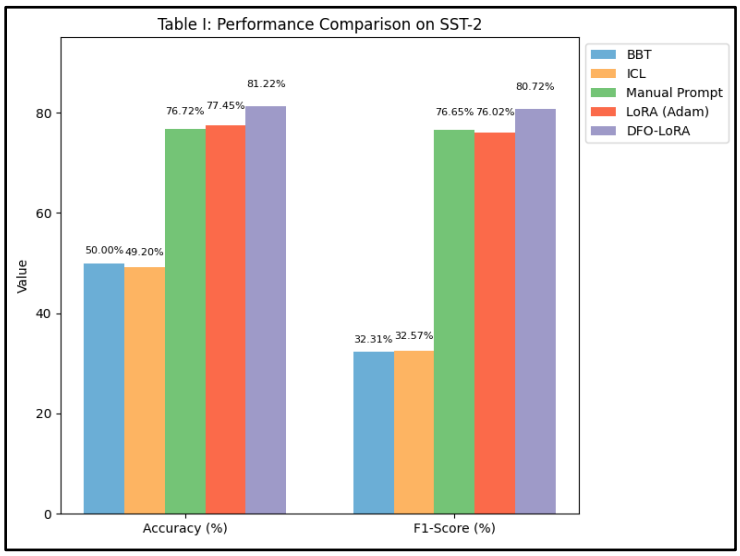


Figure 5.4 - *Performance Comparison of Different Methods on SST-2 Dataset*

Figure 5.5 shows that DFO-LoRA improves accuracy while reducing time and memory usage compared to BBT on the SST-2 dataset.

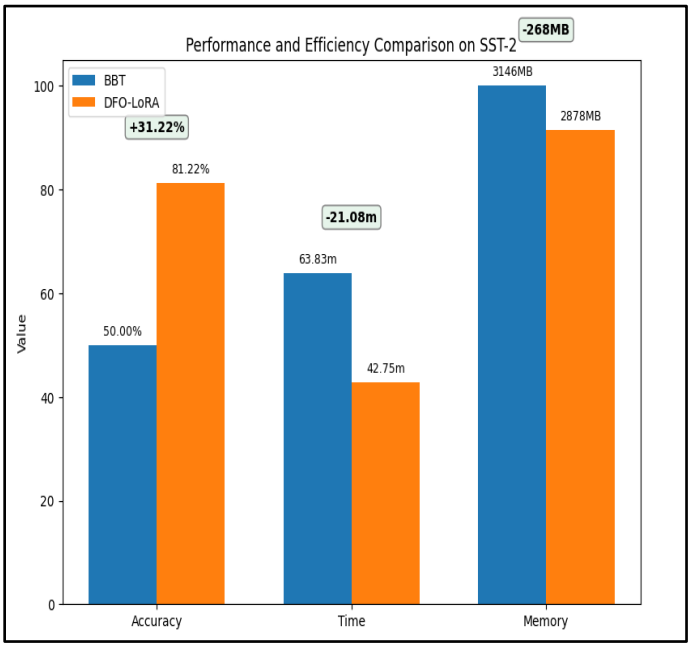


Figure 5.5 - *Performance and Efficiency Comparison between BBT and DFO-LoRA on SST-2 Dataset*

CHAPTER 6

CONCLUSION AND FUTURE WORKS

This project successfully achieved the objective of designing a parameter-efficient fine-tuning framework without relying on gradient-based optimization. By eliminating backpropagation, the proposed method significantly reduced computational overhead while maintaining strong performance, making it suitable for resource-constrained environments. The integration of low-rank adaptation modules into transformer self-attention layers was effectively implemented, enabling efficient parameter updates with a reduced number of trainable parameters.

To address the objective of proposing novel methods, C-LoRA and F-LoRA were introduced using gradient-free optimization techniques. These methods demonstrated faster convergence and effective optimization in few-shot settings.

The evaluation on several datasets confirmed that the proposed framework achieves high performance and strong generalization, with accuracy ranging from 81.22% to 88.75%, while maintaining low memory usage of 2276–2878 MB and reduced execution time. Additionally, the method outperformed gradient-based LoRA with a 3.77% improvement in accuracy, highlighting its effectiveness.

For future work, the framework can be extended to larger language models and more diverse few-shot tasks to further evaluate its robustness. Enhancements can include advanced derivative-free optimization techniques and hybrid methods combining gradient-based and gradient-free strategies. Additionally, improving convergence stability and applying the method to tasks such as text generation and summarization can further enhance its applicability in real-world scenarios.

REFERENCES

[1] S. Lin, "HybridMoE: LoRA-Based LLMs Fine-Tune With Hybrid Mixture of Experts," in IEEE Transactions on Audio, Speech and Language Processing, vol. 34, pp. 44-53, 2025, doi:10.1109/TASLPRO.2025.3637555.

[2] M. Bian, "SecLoRA: Efficient Privacy-Preserving LLM Tuning," in IEEE Internet of Things Journal, vol. 12, no. 19, pp. 40221-40234, 1 Oct.1, 2025, doi: 10.1109/JIOT.2025.3588837.

[3] S. Al-Abri, T. X. Lin, M. Tao and F. Zhang, "A Derivative-Free Optimization Method With Application to Functions With Exploding and Vanishing Gradients," in *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 587-592, April 2021, doi: 10.1109/LCSYS.2020.3004747.

[4] A. Kumar, S. Das and R. Mallipeddi, "A Reference Vector-Based Simplified Covariance Matrix Adaptation Evolution Strategy for Constrained Global Optimization," in *IEEE Transactions on Cybernetics*, vol. 52, no. 5, pp. 3696-3709, May 2022, doi: 10.1109/TCYB.2020.3013950.

[5] W. Luo, H. Jin, H. Li, X. Fang and R. Zhou, "Optimal Performance and Application for Firework Algorithm Using a Novel Chaotic Approach," in *IEEE Access*, vol. 8, pp. 120798-120817, 2020, doi:10.1109/ACCESS.2020.3004430.

[6] Mao, Y., Ge, Y., Fan, Y. A survey on LoRA of large language models. Front. Comput. Sci. 19,197605(2025).https://doi.org/10.1007/s11704-024-40663-9.

[7] R. Zhu, Z. Jiang, Z. Zhang, X. Liu, X. Liu and X. Jin, "Cannikin: No Lagger of SLO in Concurrent Multiple LoRA LLM Serving," in IEEE Transactions

on Parallel and Distributed Systems, vol. 36, no. 9, pp. 1972-1984, Sept. 2025, doi: 10.1109/TPDS.2025.3590014.

[8] Y. Peng, Y. Wang, Z. Fang, L. Zhu, Y. Deng and Y. Duan, "Revisiting LoRA: A Smarter Low-Rank Approach for Efficient Model Adaptation," 2025 5th International Conference on Artificial Intelligence and Industrial Technology Applications (AIITA), Xi'an, China, 2025, pp. 1248-1252, doi: 10.1109/AIITA65135.2025.11047778.

[9] L. Stripinis, J. Kůdela and R. Paulavičius, "Benchmarking Derivative-Free Global Optimization Algorithms Under Limited Dimensions and Large Evaluation Budgets," in IEEE Transactions on Evolutionary Computation, vol. 29, no. 1, pp. 187-204, Feb. 2025, doi: 10.1109/TEVC.2024.3379756.

[10] Z. Li, Q. Zhang, X. Lin and H. -L. Zhen, "Fast Covariance Matrix Adaptation for Large-Scale Black-Box Optimization," in IEEE Transactions on Cybernetics, vol. 50, no. 5, pp. 2073-2083, May 2020, doi:10.1109/TCYB.2018.2877641.

[11] X. Wang, X. Wang, B. Jiang and B. Luo, "Few-Shot Learning Meets Transformer: Unified Query-Support Transformers for Few-Shot Classification," in IEEE Transactions on Circuits and Systems for Video Technology, vol. 33, no. 12, pp. 7789-7802, Dec. 2023, doi: 10.1109/TCSVT.2023.3282777.

[12] J. Arabas and D. Jagodziński, "Toward a Matrix-Free Covariance Matrix Adaptation Evolution Strategy," in IEEE Transactions on Evolutionary Computation, vol. 24, no. 1, pp. 84-98, Feb. 2020, doi: 10.1109/TEVC.2019.2907266.

[13] H. Xue, "Toward Few-Shot Learning in the Open World: A Review and Beyond," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 47, no. 11, pp. 10420-10440, Nov. 2025, doi:10.1109/TPAMI.2025.3594686.

[14] A. Mussa, Z. Tuimebayev and M. Mansurova, "Make Large Language Models Efficient: A Review," in IEEE Access, vol. 13, pp. 154466-154490, 2025, doi: 10.1109/ACCESS.2025.3605110.

[15] R. P. Martinez, "Compact Model Parameter Extraction via Derivative-Free Optimization," in IEEE Access, vol. 12, pp. 123224-123235, 2024, doi: 10.1109/ACCESS.2024.3453198.